

Example classes

28 Nov. 2025
FP Seminar

- 1) Main principle of OOP: everything is an object $\rightarrow$ everything has a structure behaviour.
- 2) Typical app that manages students: $\rightarrow$ that affects via objects? other objects?

main.py

$\rightarrow$ creates python package called "testing" $\rightarrow$ tests.py

in main $\rightarrow$

```
class Test:
    def __init__(self):
        pass.
```

$\rightarrow$ place in memory where we store information that is relevant to us.

- 3) Main concept about relations: SET.

Collection vs. Set

$\downarrow$ allows duplicates. $\downarrow$ don't allow duplicates

Class $\rightarrow$ dog $\rightarrow$ structure + behaviour

Console $\rightarrow$ an object.

"-- sth" $\rightarrow$ private $\rightarrow$ "ENCAPSULATION"
self.-- commands = { }

infrastructure
Repository-Student.py
errors

exception.py

env.

main.py

presentation
console.py

business
service-stud.py

testing
tests.py

class ServiceStudents: ...

validation

Student-validator.py

U

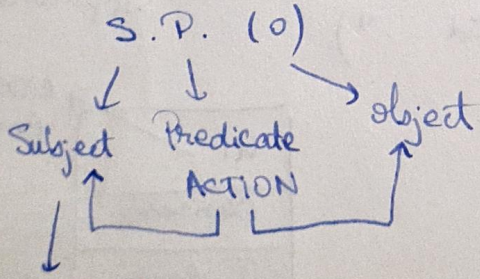
domain

Student.py

$\downarrow$ Student class

1980 OOP.

1970 Procedural Programming (C).



is an object.

with self awareness $\rightarrow$ contains a representation of themselves

René Descartes.

constructor?



Abstract Data Type → Lack of encapsulation X



Precursor of classes. → encapsulation ✓
(LOOP)

instead of x, store $f(x)$

means nothing,
it's a chipher?
only I know what is
there → apply f^{-1}

↓
x.

Student.py → getters, setters, properties etc.

class Student:

def __init__(self, id-student, name, value):

self.__id-student

self.__name

self.__value

@property

vs.

→ create a copy → nobody has direct access to the original
def name(self): return self.__name

getter

→ original information.

console.py → needs the slave

class Console:

def __init__(self, service-student)

def run(self):

while True ...

try:

self.__commands[command_name](parameters).
except ValueError("Invalid ...")

except ValueError:

raise ValueError("Invalid...")



allows us to print our own messages

(we used a class & module of errors,

as class for ... , & ~~function~~ for each error type)
classes!

